

Email Delivery Architecture and Technology Selection

1 Email Delivery Architecture

The application contains contact and inquiry forms that allow users to submit requests through the website. Whenever a user submits a form, the backend generates and sends an email notification to the administrator.

Initially, the system was designed using SMTP (Simple Mail Transfer Protocol) through Spring Boot's JavaMailSender. However, during deployment on Render, limitations related to outbound SMTP communication were encountered. To ensure reliable email delivery while minimizing infrastructure complexity, the architecture was modified to use the Resend Email API.

2 Understanding SMTP

2.1 What is SMTP?

SMTP (Simple Mail Transfer Protocol) is the standard protocol used for sending emails across the internet.

SMTP acts as the communication layer between an application and a mail server.

The typical workflow is:

Application ↓ SMTP Protocol ↓ Mail Server ↓ Recipient Inbox

Examples of SMTP providers include:

- Gmail SMTP
- Outlook SMTP
- Zoho SMTP
- Yahoo SMTP
- Self-hosted Mail Servers

2.2 SMTP in Spring Boot

SMTP integration in Spring Boot typically requires:

- SMTP Host
- SMTP Port
- Username
- Password
- TLS or SSL Configuration
- JavaMailSender

The application establishes a connection with the mail server and transmits emails using the SMTP protocol.

2.3 Advantages of SMTP

- Industry-standard protocol.
- Supported by almost every email provider.
- Provider-independent architecture.
- Can be self-hosted.
- Suitable for enterprise environments.

2.4 Limitations of SMTP

- Requires additional configuration.
- Sensitive to firewall restrictions.
- SMTP ports are frequently blocked on cloud platforms.
- Deliverability monitoring requires additional setup.
- Bounce handling is not built-in.
- More operational overhead.

3 Understanding Resend

3.1 What is Resend?

Resend is a cloud-based email delivery platform that provides email sending capabilities through APIs and SMTP interfaces.

Instead of communicating directly with an SMTP server, applications send HTTPS requests to the Resend platform.

Resend then manages email delivery on behalf of the application.

The workflow is:

Application ↓ HTTPS API Request ↓ Resend Platform ↓ Mail Servers ↓ Recipient
Inbox

3.2 Resend API Integration

The application sends email requests through the Resend REST API.

Each request includes:

- Sender Information
- Recipient Information
- Subject
- HTML Content
- Reply-To Address

The Resend platform handles the underlying email delivery process.

3.3 Advantages of Resend

- Easy API-based integration.
- No SMTP configuration required.
- Cloud-native architecture.
- Works reliably on most hosting providers.
- Built-in delivery monitoring.
- Better deliverability management.
- Simplified SPF, DKIM, and DMARC setup.
- Automatic scalability.

3.4 Limitations of Resend

- Dependency on a third-party provider.
- Paid plans beyond free-tier limits.

- Less control compared to self-hosted SMTP servers.

4 SMTP vs Resend Comparison

Table 1: Comparison between SMTP and Resend

Feature	SMTP	Resend API
Integration Complexity	Medium	Low
Cloud Deployment Compatibility	Moderate	High
Firewall Restrictions	Common	Rare
Bounce Tracking	Manual	Built-in
Monitoring and Analytics	Limited	Available
Scalability	Depends on Provider	High
Maintenance Effort	Higher	Lower
Ease of Development	Moderate	High

5 Deployment Considerations

The backend application is deployed on the Render cloud platform.

Many cloud providers restrict outbound SMTP traffic to prevent spam, abuse, and malicious email activity. As a result, direct SMTP communication can become unreliable or require additional configuration.

Resend communicates using HTTPS, which is universally supported by cloud platforms. Therefore, the application is not affected by SMTP port restrictions.

This makes Resend significantly easier to deploy and maintain in cloud-hosted environments.

6 Email Architecture Implemented in the Project

The implemented design follows a hybrid approach.

User Form Submission ↓ Email Service ↓ Check for Resend API Key ↓ Resend Available? ↓ Yes → Send via Resend No → SMTP Fallback

This design allows the system to support both email delivery mechanisms while prioritizing the more reliable cloud-native solution.

7 Justification for Choosing Resend

7.1 Project Stage Consideration

The project is currently in its early development and deployment stage.

The expected email volume primarily consists of:

- Contact Form Submissions
- Inquiry Notifications
- Lead Generation Emails

The expected traffic is relatively low and does not justify maintaining dedicated mail infrastructure.

7.2 Cost Optimization

One of the primary goals during the initial stages was to minimize infrastructure costs.

Using Resend allows the application to:

- Utilize free-tier email quotas.
- Avoid dedicated mail server costs.
- Avoid additional hosting resources.
- Reduce operational complexity.

This aligns with the objective of building a cost-efficient prototype and early-stage product.

7.3 Deployment Simplicity

Because the application is deployed on Render, using Resend avoids SMTP-related deployment issues.

The integration only requires:

- HTTPS Connectivity
- API Key Configuration

No mail server setup is necessary.

7.4 Maintainability

The implemented EmailService separates business logic from the underlying delivery mechanism.

As a result:

- Resend can be used today.
- SMTP can be enabled later.
- Alternative providers can be integrated with minimal changes.

This ensures long-term flexibility and maintainability.

8 Conclusion

SMTP remains the industry-standard protocol for email transmission and is widely used across enterprise systems. However, SMTP introduces additional configuration requirements and deployment challenges, particularly within cloud-hosted environments.

For the current stage of the project, Resend provides a simpler, more reliable, and cost-effective solution. By utilizing HTTPS-based email delivery, the application avoids SMTP restrictions while maintaining scalability and ease of deployment.

Therefore, the decision to use Resend is technically justified based on deployment constraints, cost optimization, maintainability, and cloud compatibility. Furthermore, because the architecture retains SMTP support as a fallback mechanism, the system remains flexible enough to migrate to another email delivery provider in the future if business requirements evolve.